Arrays 📚

An **array** is a fundamental linear data structure that stores a collection of elements of the same data type in contiguous memory locations. This contiguous storage allows for **O(1)** (constant time) access to any element, as the address of an element can be calculated directly using its index.

Characteristics

- **Static Size**: In many languages, arrays have a fixed size that must be declared at the time of creation. This size cannot be changed later.
- **Homogeneous Elements**: An array can only store elements of the same data type (e.g., all integers or all strings).
- **Indexed**: Each element in an array is accessed using a numerical index, typically starting from 0.

Operations and Time Complexity

- **Access**: Accessing an element at a given index is **O(1)**.
- **Insertion/Deletion**: Inserting or deleting an element in the middle of an array requires shifting all subsequent elements, making the complexity **O(n)** (linear time).
- **Search**: Searching for a specific element can be **O(n)** for a linear search or **O(log n)** for a binary search if the array is sorted.

Example: Array in C++

C++
```
#include <iostream>
using namespace std;

int main() {
    // Declare and initialize an integer array
    int arr[5] = {10, 20, 30, 40, 50};

    // Accessing an element (O(1) operation)
    cout << "Third element is: " << arr[2] << endl;

    // Insertion at the end (can be O(1) if space is available)
    // If not, it can be O(n) due to potential resizing.

    // Deletion of an element (O(n) operation)
    // To delete the element at index 2 (value 30), shift elements.
    for (int i = 2; i < 4; i++) {
        arr[i] = arr[i + 1];
    }
    cout << "Array after deleting third element: ";
    for (int i = 0; i < 4; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    return 0;
}
```

Stacks 📦

A **stack** is an abstract data type (ADT) that follows the **LIFO** (Last-In, First-Out) principle. Think of it as a stack of plates; you can only add a new plate to the top, and you can only remove the top-most plate. The primary operations are performed at one end, called the **top**.

Characteristics

- **LIFO**: The last element added to the stack is the first one to be removed.
- **Limited Access**: Only the top element can be accessed or removed.

Operations and Time Complexity

- **push(item)**: Adds an element to the top of the stack. **O(1)**.
- **pop()**: Removes and returns the top element from the stack. **O(1)**.
- **peek()/top()**: Returns the top element without removing it. **O(1)**.
- **isEmpty()**: Checks if the stack is empty. **O(1)**.

Example: Stack in Python

Python
```
# A list can be used to implement a stack in Python
stack = []

# push operation
stack.append('A')
stack.append('B')
stack.append('C')
print("Stack after push operations:", stack)

# pop operation (LIFO)
popped_item = stack.pop()
print("Popped item:", popped_item)
print("Stack after pop operation:", stack)

# peek/top operation
top_item = stack[-1]
print("Top item:", top_item)

# isEmpty check
print("Is stack empty?", not stack)
```

Queues 🚶

A **queue** is an abstract data type (ADT) that follows the **FIFO** (First-In, First-Out) principle. It's similar to a waiting line at a bank; the first person to get in line is the first person to be served. Elements are added at the **rear** and removed from the **front**.

Characteristics

- **FIFO**: The first element added to the queue is the first one to be removed.

- **Two-Ended Access**: Operations occur at both the front (dequeue) and the rear (enqueue).

Operations and Time Complexity

- **enqueue(item)**: Adds an element to the rear of the queue. **O(1)**.
- **dequeue()**: Removes and returns the element from the front of the queue. **O(1)**.
- **peek()/front()**: Returns the front element without removing it. **O(1)**.
- **isEmpty()**: Checks if the queue is empty. **O(1)**.

Example: Queue in C++

C++
```
#include <iostream>
#include <queue> // Standard library queue
using namespace std;

int main() {
    queue<int> q;

    // enqueue operation
    q.push(10);
    q.push(20);
    q.push(30);
    cout << "Queue after enqueue operations. Front element is: " <<
q.front() << endl;

    // dequeue operation (FIFO)
    q.pop();
    cout << "Queue after dequeue. New front element is: " << q.front() <<
endl;

    // peek/front operation
    cout << "Front element is: " << q.front() << endl;

    // isEmpty check
    cout << "Is queue empty? " << (q.empty() ? "Yes" : "No") << endl;

    return 0;
}
```

Linked Lists 🔗

A **linked list** is a linear data structure where elements are not stored in contiguous memory locations. Instead, each element, called a **node**, contains two parts: the **data** and a **pointer** (or reference) to the next node in the sequence.

Characteristics

- **Dynamic Size**: Linked lists can grow or shrink in size as needed.
- **Non-Contiguous Memory**: Elements are linked via pointers, so they can be scattered throughout memory.

- **Flexible Operations**: Insertion and deletion are more efficient than in arrays, as they don't require shifting elements.

Operations and Time Complexity

- **Insertion**: Inserting a node at a specific position is typically **O(1)** after finding the insertion point. Finding the position can be **O(n)**.
- **Deletion**: Deleting a node is typically **O(1)** after finding the node to be deleted. Finding the node can be **O(n)**.
- **Access/Search**: To access or search for an element at a specific index, you must traverse the list from the beginning, resulting in **O(n)** complexity.

Example: Linked List in C++

C++
```
#include <iostream>
using namespace std;

// Node structure
class Node {
public:
    int data;
    Node* next;

    // Constructor to create a new node
    Node(int data) {
        this->data = data;
        this->next = nullptr;
    }
};

int main() {
    // Create the first node (head)
    Node* head = new Node(10);
    Node* second = new Node(20);
    Node* third = new Node(30);

    // Link the nodes together
    head->next = second;
    second->next = third;

    // Traversal to print the list
    Node* current = head;
    cout << "Linked list elements: ";
    while (current != nullptr) {
        cout << current->data << " ";
        current = current->next;
    }
    cout << endl;

    return 0;
}
```